DSA & Advanced DSA Roadmap

Phase 5 — Queue

Learning Objectives

By the end of this phase, learners will be able to:

- Explain the FIFO (First-In-First-Out) principle and implement a queue using both an array and a linked list.
- Implement a circular queue and a double-ended queue (deque), and explain why circular indexing avoids wasted space.
- Connect the queue data structure to Breadth-First Search (BFS) and explain why BFS requires a queue rather than a stack.
- Identify and apply core queue patterns: BFS foundations, sliding window maximum, monotonic queues, and multi-source BFS.
- Solve Easy to Medium level queue problems independently, and approach Hard sliding-window problems using the monotonic deque technique.

Implementation & Internals

Core Concepts

- Queue — implementing enqueue/dequeue backed by an array or linked list, and why a naive array-shifting implementation costs O(n) per dequeue.
- Circular Queue — using modulo arithmetic on a fixed-size array so the queue wraps around, reclaiming space freed by dequeues without shifting elements.
- Deque (Double-Ended Queue) — supporting O(1) insertion and removal at both ends, and why this generality makes it the backbone of monotonic queue techniques.
- Priority Queue Basics — a conceptual introduction only; full heap-backed priority queue implementation is covered in Phase 12 (Heap).

Implementation Exercises

- Build a queue from scratch using a dynamic array, supporting enqueue, dequeue, peek, and isEmpty, without using array.shift() or equivalent O(n) operations.
- Build a circular queue backed by a fixed-size array using head/tail indices and modulo arithmetic.
- Build a deque from scratch supporting push/pop at both the front and back.
- Implement a basic BFS traversal of a small graph or grid using your own queue, and trace how the queue's contents change level by level.

Pattern Mastery

Pattern 1: BFS Foundation

Concepts

- Using a queue to explore nodes level by level, guaranteeing the shortest path is found first in an unweighted graph or grid.
- Tracking visited nodes to avoid revisiting and infinite loops.
- Processing level-by-level by tracking the queue's size at the start of each level.

Practice Problems

- Traverse a binary tree level by level, returning each level as its own list.
- Implement a moving average over a stream of integers using a queue-backed sliding window.

LeetCode

- Binary Tree Level Order Traversal — leetcode.com/problems/binary-tree-level-order-traversal
- Moving Average from Data Stream — leetcode.com/problems/moving-average-from-data-stream

Pattern 2: Sliding Window Maximum

Concepts

- Finding the maximum (or minimum) value in every fixed-size window as it slides across an array.
- Why a naive per-window scan costs O(n·k), and why a monotonic deque reduces this to O(n).

Practice Problems

- Given an array and window size k, return the maximum value in each window as it slides from left to right.

LeetCode

- Sliding Window Maximum — leetcode.com/problems/sliding-window-maximum

Note: this is a Hard problem placed here deliberately as the canonical exercise for this pattern; it reappears in the Phase Assessment's Hard tier as the same problem, not a different one.

Pattern 3: Monotonic Queue

Concepts

- Maintaining a deque whose elements are kept in increasing or decreasing order by popping from the back before each push.
- Why the front of a monotonic decreasing deque always holds the current window's maximum.
- Distinguishing a monotonic queue (this phase, using a deque) from a monotonic stack (Phase 4).

Practice Problems

- Implement push/pop operations for a deque that maintains monotonic order, as a standalone exercise before applying it to Sliding Window Maximum.
- Support push, pop, and getMiddle-style queries on a queue that allows insertion and removal at front, middle, and back.

LeetCode

- Design Front Middle Back Queue — leetcode.com/problems/design-front-middle-back-queue

Pattern 4: Multi-Source BFS

Concepts

- Seeding a BFS queue with multiple starting points simultaneously, rather than running BFS once per source.
- Why seeding all sources at once (rather than looping BFS per source) avoids redundant work and gives the correct minimum distance to the nearest source for every cell.
- Recognizing the 'multi-source BFS' shape: a grid where several cells start as sources and the answer is the shortest distance (or time) to spread to every other cell.

Practice Problems

- Given a grid where some cells start 'rotten', find the minimum time for all reachable cells to become rotten, started from every rotten cell at once.
- Given a binary matrix, find the distance from each cell to its nearest 0, starting BFS from every 0 cell simultaneously.

LeetCode

- Rotting Oranges — leetcode.com/problems/rotting-oranges
- 01 Matrix — leetcode.com/problems/01-matrix

Phase Assessment

Implementation Tasks

- Build a queue from scratch using a dynamic array, without relying on O(n) shift operations.
- Build a circular queue from scratch using a fixed-size array and modulo arithmetic.
- Analyze and explain why multi-source BFS gives the correct minimum distance for every cell in a single pass, rather than requiring one BFS per source.

Recommended LeetCode Target

Easy Problems

- leetcode.com/problems/binary-tree-level-order-traversal
- leetcode.com/problems/moving-average-from-data-stream
- leetcode.com/problems/implement-queue-using-stacks

Medium Problems

Implementation (Design)

- leetcode.com/problems/design-circular-queue
- leetcode.com/problems/design-circular-deque
- leetcode.com/problems/design-front-middle-back-queue

Multi-Source BFS

- leetcode.com/problems/rotting-oranges
- leetcode.com/problems/01-matrix

Hard Problems

- leetcode.com/problems/sliding-window-maximum

On scope and dedup

This phase has 9 distinct problems, larger than the Phase 1–3 baseline. Number of Islands and Walls and Gates were deliberately excluded: Number of Islands is reserved for Phase 14 (Graphs) as the canonical connected-components problem, and Walls and Gates (LeetCode 286) requires LeetCode Premium, so 01 Matrix is used instead as the free, equivalent multi-source BFS exercise. None of the 9 problems overlaps with any problem used in Phases 0–4.

Coding Challenges (Assessment Day)

- Binary Tree Level Order Traversal
- Design Circular Queue
- Rotting Oranges
- 01 Matrix
- Sliding Window Maximum

Expected Outcome

Students should be able to:

- Implement a queue, circular queue, and deque from scratch, and explain why each design choice avoids unnecessary O(n) operations.
- Recognize common queue patterns: BFS foundations, sliding window maximum, monotonic queues, and multi-source BFS.
- Solve unseen queue and grid-BFS problems using pattern-based thinking rather than memorized solutions.
- Complete Easy and Medium queue questions independently, and approach Hard sliding-window problems using the monotonic deque technique introduced this phase.